

Data with Anurag

Python Interview Practice for Data Engineers

This practice sheet is designed for candidates preparing for Python rounds in Data Engineering interviews.

The focus is on problem-solving, data handling, file processing, APIs, dictionaries, exceptions, and memory-efficient coding.

Try solving every question on your own before checking any reference solution.

1. Find Duplicate Records

You receive customer IDs from a source system.

```
customer_ids = [101, 102, 103, 101, 104, 105, 102, 106]
```

Task

Find all duplicate customer IDs.

Expected Output

```
{101, 102}
```

Interview Follow-up

- Can you solve this without using `count()` repeatedly?
 - Why is a set useful here?
 - What is the time complexity of your approach?
-

2. Find the Latest Record Per Customer

You receive multiple versions of customer records.

```
records = [  
  {"customer_id": 101, "city": "Delhi", "updated_at": "2025-01-01"},  
  {"customer_id": 101, "city": "Bengaluru", "updated_at": "2025-01-05"},  
  {"customer_id": 102, "city": "Mumbai", "updated_at": "2025-01-03"},  
  {"customer_id": 102, "city": "Pune", "updated_at": "2025-01-07"}  
]
```

Task

Keep only the latest record for every customer.

Expected Output

Customer 101 → Bengaluru

Customer 102 → Pune

Interview Follow-up

- Would you compare timestamps as strings in production?
 - How would you handle multiple records with the same timestamp?
 - How would this logic change for millions of records?
-

3. Flatten Nested API Response

An API returns:

```
data = {  
  "customer_id": 101,  
  "customer_name": "John",  
  "orders": [  
    {"order_id": 1, "amount": 500},  
    {"order_id": 2, "amount": 800},  
    {"order_id": 3, "amount": 1200}  
  ]  
}
```

Task

Convert this nested structure into:

```
[
  {"customer_id": 101, "customer_name": "John", "order_id": 1, "amount": 500},
  {"customer_id": 101, "customer_name": "John", "order_id": 2, "amount": 800},
  {"customer_id": 101, "customer_name": "John", "order_id": 3, "amount": 1200}
]
```

Interview Follow-up

- How would you handle missing `orders`?
 - What if the API returns multiple customers?
 - How would you process paginated API responses?
-

4. Separate Good and Bad Records

Input:

```
values = ["100", "250", "abc", "500", "xyz", "700"]
```

Task

Convert valid numeric values into integers.

Store invalid records separately.

Expected Output

```
clean = [100, 250, 500, 700]
```

```
bad = ["abc", "xyz"]
```

Interview Follow-up

- Which exception would you catch?
 - Why should you avoid a generic `except :?`
 - How would you log rejected records in a real pipeline?
-

5. Compare Two Daily Snapshots

Yesterday:

```
yesterday = [101, 102, 103, 104, 105]
```

Today:

```
today = [101, 103, 104, 105, 106]
```

Task

Find:

- New customers
- Deleted customers
- Existing customers

Expected Result

New: 106

Deleted: 102

Existing: 101, 103, 104, 105

Interview Follow-up

- Which Python data structure makes this easy?
 - How would you do this for dictionaries instead of IDs?
 - Where is this pattern useful in Data Engineering?
-

6. Aggregate Revenue Per Customer

Input:

```
orders = [  
    {"customer_id": 101, "amount": 500},  
    {"customer_id": 102, "amount": 700},  
    {"customer_id": 101, "amount": 300},
```

```
{ "customer_id": 103, "amount": 1000},  
{ "customer_id": 102, "amount": 200}  
]
```

Task

Calculate total revenue for each customer.

Expected Output

```
{  
  101: 800,  
  102: 900,  
  103: 1000  
}
```

Interview Follow-up

- Can you solve this using `defaultdict`?
 - Can you solve it using `Counter`?
 - Which approach is more readable?
-

7. Top 3 Customers by Revenue

Using the aggregated revenue from the previous problem:

Task

Return the top 3 customers based on total revenue.

Interview Follow-up

- How would you sort a dictionary?
 - What if there are 10 million customers?
 - Would sorting the complete dataset always be efficient?
-

8. Join Two Datasets Without Pandas

Customer master:

```
customers = [  
    {"customer_id": 101, "name": "John"},  
    {"customer_id": 102, "name": "Emma"},  
    {"customer_id": 103, "name": "David"}  
]
```

Orders:

```
orders = [  
    {"order_id": 1, "customer_id": 101, "amount": 500},  
    {"order_id": 2, "customer_id": 102, "amount": 700},  
    {"order_id": 3, "customer_id": 104, "amount": 900}  
]
```

Task

Join orders with customer details.

For missing customers, use "Unknown".

Interview Follow-up

- How would you optimize the lookup?
 - Why is creating a dictionary from customer master useful?
 - What SQL join does this resemble?
-

9. Process a Large File Efficiently

You receive a 20 GB log file.

Task

Explain how you would process it using Python without loading the complete file into memory.

Interview Expectation

The candidate should discuss:

- Reading line by line
- Generators
- Context managers
- Chunk processing

Follow-up

Why is this bad?

```
data = file.read()
```

for a very large file?

10. Generator-Based Batch Processing

Task

Write a generator that produces records in batches of 1000.

Example input:

```
records = range(1, 10001)
```

Requirement

The function should return one batch at a time instead of creating all batches in memory.

Interview Follow-up

- Difference between `return` and `yield`
 - Why generators are memory efficient
 - Where generators are useful in Data Engineering
-

11. Handle API Pagination

An API returns 100 records per page.

You need to retrieve all available records.

Task

Design a Python loop that:

- Calls the API
- Reads one page
- Stores/processes records
- Moves to the next page
- Stops when no more records are returned

Interview Follow-up

What production features would you add?

Expected points:

- Retry
 - Timeout
 - Authentication
 - Logging
 - Rate-limit handling
 - Exponential backoff
-

12. Retry a Failed API Call

Scenario

An API sometimes returns temporary errors.

Task

Implement retry logic with maximum 3 attempts.

Interview Follow-up

- Should every error be retried?
 - What is exponential backoff?
 - What HTTP errors are generally retryable?
-

13. Parse a Large JSON File

Scenario

A large JSON file contains customer transactions.

Task

Explain how you would process the file when it is too large to fit in memory.

Interview Discussion

Think about:

- Streaming JSON
 - Line-delimited JSON
 - Chunking
 - Avoiding full-file loading
-

14. Clean Customer Data

Input:

```
customers = [  
  {"id": 1, "name": "john", "age": "25"},  
  {"id": 2, "name": "EMMA", "age": "abc"},  
  {"id": 3, "name": None, "age": "30"},  
  {"id": 1, "name": "john", "age": "25"}  
]
```

Requirements

- Remove duplicate IDs

- Standardize names
- Convert age into integer
- Reject invalid ages
- Replace missing names with "Unknown"

Interview Follow-up

- In what order should these transformations happen?
 - Would you silently discard bad records?
 - How would you maintain rejected records?
-

15. Group Records by Key

Input:

```
transactions = [  
    ("Bengaluru", 100),  
    ("Delhi", 200),  
    ("Bengaluru", 300),  
    ("Mumbai", 150),  
    ("Delhi", 250)  
]
```

Task

Create:

```
{  
    "Bengaluru": [100, 300],  
    "Delhi": [200, 250],  
    "Mumbai": [150]  
}
```

Follow-up

Try solving using:

- Normal dictionary
- `defaultdict`

16. Find Missing Sequence Numbers

Input:

```
ids = [1001, 1002, 1004, 1005, 1008]
```

Task

Find missing IDs between minimum and maximum.

Expected Output

```
[1003, 1006, 1007]
```

Interview Follow-up

Where could this be useful?

- Data quality
- Missing ingestion batches
- Sequence validation

17. Remove Duplicate Dictionaries

Input:

```
records = [  
    {"id": 1, "name": "John"},  
    {"id": 2, "name": "Emma"},  
    {"id": 1, "name": "John"},  
    {"id": 3, "name": "David"}  
]
```

Task

Remove exact duplicate records.

Follow-up

What changes if duplicate is defined only by `id`?

18. Sort Records Using Multiple Conditions

Input:

```
employees = [  
    {"name": "John", "department": "IT", "salary": 90000},  
    {"name": "Emma", "department": "HR", "salary": 70000},  
    {"name": "David", "department": "IT", "salary": 100000},  
    {"name": "Sophia", "department": "HR", "salary": 80000}  
]
```

Task

Sort by:

1. Department ascending
2. Salary descending

Interview Follow-up

Explain how a `lambda` function works here.

19. Create a Data Quality Summary

Input:

```
records = [  
    {"id": 1, "email": "a@gmail.com", "age": 25},  
    {"id": 2, "email": None, "age": 30},  
]
```

```
{ "id": 3, "email": "c@gmail.com", "age": -5},  
{ "id": 4, "email": None, "age": None}  
]
```

Task

Calculate:

- Total records
- Null emails
- Null ages
- Invalid ages
- Valid records

Interview Follow-up

How would you convert this into a reusable validation function?

20. Build a Simple ETL Function

Scenario

Input records come from an API.

Task

Create a reusable Python function that:

1. Accepts raw records
2. Removes duplicates
3. Validates mandatory fields
4. Cleans strings
5. Separates rejected records
6. Returns clean and rejected datasets

Interview Expectation

Do not only focus on getting the output.

Think about:

- Function design
 - Error handling
 - Logging
 - Reusability
 - Testability
-

Important Python Topics to Revise for Data Engineering Interviews

Focus strongly on:

- Lists, tuples, sets and dictionaries
 - List and dictionary comprehensions
 - Functions
 - Lambda functions
 - Exception handling
 - File handling
 - Context managers
 - JSON processing
 - API handling
 - Iterators and generators
 - `collections` module
 - Sorting
 - String manipulation
 - Date/time handling
 - Basic OOP
 - Memory-efficient processing
-

Quick Interview Advice

Do not directly start writing code.

First clarify:

- What is the input?
- What is the expected output?
- Can duplicates exist?

- Can null values exist?
- How large can the data become?
- What should happen to invalid records?

In Data Engineering interviews, these questions often matter as much as the final Python code.

Data with Anurag

Practice the questions by writing the code yourself first.

Avoid memorizing solutions.

For each problem, try to explain:

Approach → Code → Complexity → Edge Cases → Production Use

That will make your Python interview preparation much stronger.

DataX Bootcamp

For structured Data Engineering preparation covering SQL, Python, PySpark, Databricks, projects, cloud, and interview preparation:

DataX Bootcamp – Zero to Job Ready AI-Enabled Data Engineer

<https://login.datawithanurag.com/services/dataxbootcamp>

WhatsApp Support:

<https://wa.me/918957284012>